ADVANCED WEB DEVELOPMENT

PROJECT PLAN

CS3821 – Final Year Project

STUDENT NAME: Olukorede Oduniyi
SUPERVISOR: Susnas Sourjah

BSC (HONS) IN COMPUTER SCIENCE (SOFTWARE ENGINEERING)
DEPARTMENT OF COMPUTER SCIENCE
ROYAL HOLLOWAY, UNIVERSITY OF LONDON

2025

Contents

Chapter 1

Abstract

Asset management is, and always has been, central to everyday life, being crucial for wealth creation, risk mitigation, and achieving long-term financial goals [5]. From shares and cryptocurrencies to property, commodities, and even digital services, the ability to oversee one's assets effectively is fundamental to financial security and long-term growth. In an increasingly fast-moving world, where markets shift in seconds and opportunities can be won or lost just as quickly, the need for clear, consolidated oversight has never been greater. Without it, individuals risk missed opportunities, poor decisions, and diminished wealth. The current landscape, however, is fragmented. Investors and everyday users alike are forced to juggle multiple platforms: one to track stocks, another for cryptocurrencies, separate tools for property valuations, and yet more for commodities or subscription services. This disjointed approach makes asset management cumbersome, inefficient, and prone to error. Despite the importance of having a unified view of wealth, there remains no comprehensive solution that seamlessly integrates every type asset into a single system.

This project seeks to address that gap by developing a scalable, web-based platform capable of live tracking across diverse asset classes. Beginning with stocks and cryptocurrencies, the system is designed to expand towards property, commodities, and beyond—ultimately offering users a central, intuitive dashboard for all their financial and digital assets.

At present, users are forced to rely on a range of different platforms to monitor their assets. One application may allow them to follow shares, another tracks cryptocurrencies, while property, commodities, or digital services are often left out entirely. This fragmented approach makes it unnecessarily difficult to gain a clear, unified view of personal wealth. The lack of integration also creates inefficiency. Calculating the total value of one's holdings across different asset classes requires switching between apps, exporting data, or carrying out manual calculations. This not only wastes time but also increases the likelihood of mistakes, making effective asset management harder than it needs to be.

This project proposes the development of a web-based platform that unifies asset management across multiple classes into a single, accessible space. The platform will be built using **React JS** for the frontend interface and **Java Spring** for the backend logic and data management, utilizing **PostgreSQL** as the core relational database [4]. Users will be able to add and track stocks, cryptocurrencies, and commodities [2], with real-time market data providing up-to-date valuations. Unlike existing tools that focus on individual asset types, this system offers a complete and consolidated view of personal wealth. For financial assets, the platform allows users to input purchase prices and quantities to automatically calculate gains or losses. Property tracking presents a unique challenge, as these assets are illiquid and lack a single market price [3]. For property, tailored inputs will capture purchase price, mortgage details, and rental income, enabling long-term performance tracking. All this information will be combined into a clear, real-time portfolio overview. Future developments include integration of local rental market data, economic news linked directly to relevant assets, and expansion into further asset classes. The overall aim is to deliver a comprehensive, intelligent tool that simplifies asset management and empowers users to make better-informed financial decisions.

The goal of this project is to create a simple, user-friendly platform for managing multiple asset classes in one place. The website will feature a dashboard where users can view a complete list of their assets, including stocks, cryptocurrencies, commodities, and property. Each entry will display key details such as purchase price, quantity, and current value. A central feature of the dashboard will be a large, interactive graph that visualises the performance of all assets together, enabling users to see overall portfolio trends alongside individual asset movements. Real-time data feeds will ensure valuations

are always up to date, while built-in calculations will automatically track gains, losses, and long-term performance. The system is designed to be scalable, allowing new asset types and features—such as local property data or economic indicators—to be added over time, making the platform future-proof and adaptable to user needs.

This project offers a genuine benefit to all users by simplifying the way assets are tracked and understood. By bringing stocks, cryptocurrencies, commodities, and property together into one unified dashboard, it removes the need to juggle multiple apps and calculations. Even the average person, with only a few assets, gains clear visibility of their financial position and long-term growth. With real-time data and intuitive visualisation, the platform empowers users to make informed decisions, avoid costly mistakes, and feel more in control of their financial future.

Chapter 2

Timeline

By the end of Term 1, the aim is to have the fundamental system in place with full end-to-end functionality, including stock additions, charting features, and completion of the interim report.

2.1 Term 1

- **Week 1:** Research and planning for the project.
- **Week 2:** Frontend and backend design, along with database architecture.
- **Week 3-4:** Database created with models and a simple homepage design.
- **Week 5-6:** Addition of stocks, cryptocurrencies, and commodities. Implementation of RESTful API endpoints in Java Spring to connect the PostgreSQL database models to the React JS frontend.
- **Week 7:** Research on integrating charts and live tracking into the frontend.
- **Week 8-9:** Development of charts and dashboard, including unit and integration testing, bug fixing, and initial user acceptance testing.
- **Week 10-11:** Preparation of interim report and presentation.

Chapter 3

Risks and Mitigations

This web-based asset management platform faces risks such as security breaches, performance issues, browser compatibility problems, and potential project delays. These will be mitigated not only through secure coding, regular testing, performance optimisation, cross-browser checks, and careful planning, but also by following robust software engineering practices. Approaches such as test-driven development (TDD), version control, automated testing, continuous integration and deployment (CI/CD), code reviews, agile methodologies, and user-focused design will help ensure the platform is high-quality, reliable, and maintainable throughout development.

3.1 Security Breaches

As this project involves tracking sensitive financial data, there is a risk of security breaches, including unauthorised access, data leaks, or manipulation of user information. Such breaches could compromise user trust and the integrity of the platform. To mitigate this risk, all code and data will follow secure coding practices, including input validation and encryption for sensitive information. The project will also use regular security testing, code reviews, and automated vulnerability checks to identify and resolve issues early. Continuous integration pipelines will run automated security and unit tests on every commit, ensuring potential flaws are caught before deployment. Additionally, user authentication and access controls will be implemented, and all work will be stored in secure cloud repositories such as Gitlab, with version control ensuring safe collaboration and rollback if necessary. By proactively addressing these risks, the platform aims to maintain a reliable, safe, and trustworthy environment for asset management.

3.2 Performance Issues

The platform may experience performance problems, particularly under high user traffic or when processing real-time asset data. Slow response times or system crashes could hinder usability and reduce user satisfaction. To mitigate this risk, code will be optimised from the outset, with careful attention to algorithmic efficiency and resource management. Regular load testing and automated performance benchmarks will be conducted to identify bottlenecks. CI/CD pipelines will integrate these tests into the build process, ensuring performance issues are identified and resolved continuously. Agile iteration cycles will also make it easier to monitor, evaluate, and improve performance in response to user feedback. Handling high-frequency, real-time data streams inherently requires specialised infrastructure, making robust performance planning essential.

3.3 Browser Compatibility

There is a risk that certain features of the platform may not function consistently across different browsers or devices, potentially restricting user access or causing interface issues. This will be mitigated through responsive design practices, cross-browser automated testing, and adherence to web standards. Continuous integration will ensure compatibility tests run automatically as the system evolves, catching regressions before they impact users. Any inconsistencies identified during testing will be corrected immediately, ensuring a seamless user experience across all platforms.

3.4 Project Delays

Delays can occur due to unexpected technical challenges, the integration of new features, or underestimation of task complexity. Such delays may impact the timely delivery of both the code and the accompanying report. To mitigate this, all tasks will be broken into manageable sub-tasks with realistic time estimates. Agile project management methodologies, such as iterative sprints and regular progress reviews, will help keep development on track and responsive to challenges. Version control will support collaborative, incremental progress, while CI/CD pipelines will automate deployment and testing to avoid wasted time on manual processes. Test-driven development (TDD) will help ensure that functionality is built correctly from the start, reducing rework and preventing delays caused by bugs in critical areas.

Bibliography

[1] Syncfusion. (2025). *React Stock Chart - Visualize Market Trends with Ease*. [Online]. Available at: `https://www.syncfusion.com/react-components/react-stock-chart` (Accessed: 7 October 2025).

[2] Bloomberg. (2025). *Commodities Market Data Overview*. [Online]. Available at: `https://www.bloomberg.com/markets/commodities` (Accessed: 7 October 2025).

[3] Zillow. (2025). *How Zillow's Zestimate works*. [Online]. Available at: `https://www.zillow.com/zestimate/` (Accessed: 7 October 2025).

[4] PostgreSQL Global Development Group. (2025). *PostgreSQL: The World's Most Advanced Open Source Relational Database*. [Online]. Available at: `https://www.postgresql.org/` (Accessed: 9 October 2025).

[5] Investopedia. (2025). *What Is Asset Management, and What Do Asset Managers Do?*. [Online]. Available at: `https://www.investopedia.com/terms/a/assetmanagement.asp` (Accessed: 9 October 2025).

[6] Tambavekar, V. (2025). *Importance of Spring Boot in FinTech and Banking Services*. [Online]. Medium. Available at: `https://medium.com/@vtambavekar239/importance-of-spring-boot-in-fintech-and-banking-services-0412f45727f2` (Accessed: 6 October 2025).

[7] Capital Numbers. (2025). *Virtual DOM in React: Concepts, Benefits, and Examples*. [Online]. Available at: `https://www.capitalnumbers.com/blog/virtual-dom-in-react/` (Accessed: 20 May 2025).